

Memory Hierarchy

Computer Organization & Architecture - Interview & Placement Guide Covers: registers, cache, RAM, storage, locality, cache mapping, replacement policies, write policies, miss types, multi-level caches, cache coherence, memory interleaving, virtual memory, NUMA, and hardware prefetching.

1. Overview

Definition

The **memory hierarchy** is the layered arrangement of storage in a computer, ordered by **speed, cost per byte, and capacity**. Fast, small, expensive memory sits close to the CPU; slow, large, cheap memory sits far away.

```
Fastest / Smallest / Costliest
Registers
L1 Cache
L2 Cache
L3 Cache
Main Memory (RAM / DRAM)
SSD / HDD (Storage)
Tape / Cloud / Archival
Slowest / Largest / Cheapest
```

The core problem it solves

There is a fundamental tension:

- **Fast memory** (SRAM, registers) is **expensive** and **small**.
- **Cheap memory** (DRAM, disk) is **large** but **slow**.

A CPU running at ~3-4 GHz completes an instruction in well under a nanosecond, but a DRAM access takes ~50-100 ns. If the CPU waited for main memory on every access, it would stall for **hundreds of cycles**. The hierarchy hides this gap by keeping frequently used data close.

The whole trick works because of **locality of reference** (Section 2). Without locality, a hierarchy would give almost no benefit.

Why it matters

- **Performance:** Real program speed is dominated by memory access patterns, not raw CPU speed. A cache miss can cost 100x more than a hit.
- **Cost efficiency:** You get the *illusion* of a large, fast memory at the price of a mostly-cheap one.
- **The "memory wall":** CPU speed has historically grown far faster than memory speed, so the hierarchy is what keeps modern CPUs fed.

Where it is used in real systems

Layer	Real-world analogue
CPU cache	Intel/AMD/ARM chips, every phone and laptop
RAM	OS page cache, in-memory databases (Redis)
Storage	Databases, file systems, virtual memory swap
Distributed	CDN edge caches, browser cache, Redis in front of Postgres

The **same hierarchy idea** repeats at every scale: CPU cache → RAM, RAM → disk, disk → network, edge cache → origin server. Learn it once at the hardware level and you understand caching everywhere.

Why interviewers ask about it

- It tests whether you understand **why code is fast or slow**, not just whether it's correct.
- It connects hardware to real performance tuning (data structure layout, loop order, false sharing).
- Questions scale from "what is a cache hit" (basic) to "explain MESI and false sharing" (senior).
- It shows up in **systems design**, **low-latency engineering**, and **performance debugging** roles.

2. Core Idea

Intuition

Keep the data you're **likely to use next** as close to the CPU as possible. Instead of predicting the future perfectly, hardware bets on a simple, reliable pattern: **programs tend to reuse recent data and access nearby data**. That bet is called locality, and it almost always pays off.

Real-world analogy: The desk, the drawer, and the warehouse

Imagine you're doing research:

- **Registers** = the few papers in your hands right now.
- **Cache** = the papers on your desk - a handful you reach for constantly.
- **RAM** = the filing cabinet drawer beside you - takes a few seconds to open.
- **Storage (disk)** = the warehouse across town - takes hours to fetch from.

You don't walk to the warehouse for every fact. You pull a batch of related files onto your desk (spatial locality) and keep the ones you keep re-reading right in front of you (temporal locality). The hierarchy is exactly this, automated in hardware.

Small example

```
int sum = 0;
int arr[1000];
for (int i = 0; i < 1000; i++)
    sum += arr[i];           // arr[i] accessed sequentially
```

- First access to `arr[0]` **misses** the cache → hardware loads a whole **cache line** (say 64 bytes = 16 ints) from RAM.
- `arr[1]..arr[15]` are now already in cache → **hits** (spatial locality).
- `sum` is accessed every iteration and lives in a **register** (temporal locality, kept super close).

Result: ~1 miss per 16 elements instead of 1000 misses. This is why the loop is fast.

Step-by-step: what happens on a memory access

1. CPU needs data at address `X`.
2. Check **L1 cache**. Hit? Return in ~1-4 cycles. Done.
3. Miss? Check **L2** (~10-15 cycles). Hit? Return, and copy the line into L1.
4. Miss? Check **L3** (~40 cycles). Then **RAM** (~100-300 cycles). Then **disk** (millions of cycles) if the page was swapped out.
5. When data arrives, it's brought in as a **whole cache line**, and something may be **evicted** to make room (replacement policy).

Each level acts as a **filter**: most accesses are caught early, so the average access time stays close to L1 speed.

The key formula: Average Memory Access Time (AMAT)

$$\text{AMAT} = \text{Hit time} + \text{Miss rate} \times \text{Miss penalty}$$

For multiple levels this nests:

$$\text{AMAT} = \text{L1_hit} + \text{L1_miss_rate} \times (\text{L2_hit} + \text{L2_miss_rate} \times (\text{L3_hit} + \text{L3_miss_rate} \times \text{MemLatency}))$$

This one formula is the quantitative heart of the whole topic. Interviewers love making you compute it.

Worked example:

- L1 hit = 1 cycle, L1 miss rate = 5%
- L2 hit = 12 cycles, L2 miss rate = 20% (of L1 misses)
- Memory = 200 cycles

$$\begin{aligned} \text{AMAT} &= 1 + 0.05 \times (12 + 0.20 \times 200) \\ &= 1 + 0.05 \times (12 + 40) \\ &= 1 + 0.05 \times 52 \\ &= 1 + 2.6 = 3.6 \text{ cycles} \end{aligned}$$

Even with a slow 200-cycle memory, AMAT is only 3.6 cycles because most accesses hit early.

3. Important Subtopics

3.1 Registers, Cache, RAM, and Storage

What it means: The physical levels of the hierarchy and their characteristics.

Level	Technology	Typical Size	Latency (approx)	Managed by
Registers	Flip-flops in CPU	~1-4 KB total	<1 cycle	Compiler
L1 Cache	SRAM	32-64 KB (per core)	~1-4 cycles	Hardware
L2 Cache	SRAM	256 KB-1 MB (per core)	~10-15 cycles	Hardware
L3 Cache	SRAM	8-64 MB (shared)	~40 cycles	Hardware
RAM	DRAM	8-128 GB	~100-300 cycles (~60-100 ns)	OS + MMU
SSD	NAND Flash	256 GB-4 TB	~10-100 μ s	OS / FS
HDD	Magnetic disk	1-20 TB	~5-10 ms	OS / FS

Why it matters: Each step down is roughly **10-100x slower** but much larger. The jump from cache to RAM, and RAM to disk, is where the biggest stalls happen.

SRAM vs DRAM (a favorite interview point):

- **SRAM** (cache): 6 transistors per bit, fast, no refresh needed, expensive, low density.
- **DRAM** (main memory): 1 transistor + 1 capacitor per bit, slower, needs periodic **refresh** (capacitor leaks), cheap, high density.

Example: A register access is essentially free; an L1 hit costs a couple cycles; a full RAM access on an L3 miss can cost 200+ cycles, during which the CPU could have executed hundreds of instructions.

Interview angle: "Order these by speed and explain why registers are fastest." (Answer: on-chip, no addressing/lookup, directly wired into the ALU datapath.)

3.2 Locality of Reference

What it means: The empirical observation that memory accesses are **not random** - they cluster in time and space. This is the *reason* caching works at all.

Two forms: temporal and spatial (detailed in 3.3 and 3.4).

Why it matters: If accesses were truly random across a 16 GB address space, a 32 KB cache would almost never hit. Locality is what makes the hierarchy effective.

Example: Loops (reuse the same code and counters), arrays (sequential), and stack variables (reused constantly) all exhibit strong locality.

Interview angle: "Why does a cache help?" → Answer with locality. If you can't cite locality, you don't understand the foundation.

3.3 Temporal Locality

What it means: If you access a memory location, you'll likely access it **again soon** (reuse over *time*).

Why it matters: It justifies *keeping* recently used data in fast memory rather than evicting immediately.

Example:

```
for (int i = 0; i < n; i++)
    total += arr[i];    // 'total' and 'i' reused every iteration
```

`total` and the loop counter `i` are accessed thousands of times → kept in registers/L1.

Common interview angle: "Give an example of temporal locality." Loop variables, accumulators, frequently called functions (instruction cache), hot config objects.

3.4 Spatial Locality

What it means: If you access a location, you'll likely access **nearby** locations soon (reuse over *space*).

Why it matters: It justifies fetching a **whole block/cache line** at once instead of a single byte. One miss "prefetches" the neighbors for free.

Example:

```
for (int i = 0; i < n; i++)
    sum += arr[i];    // arr[i], arr[i+1] ... are contiguous
```

Also: struct fields accessed together, sequential file reads.

Classic contrast - row-major vs column-major traversal:

```
// GOOD: row-major access (C stores rows contiguously) - strong spatial locality
for (i) for (j) a[i][j]++;

// BAD: column-major access - jumps by a full row each step, cache-unfriendly
for (j) for (i) a[i][j]++;    // can be 5-10x slower on large matrices
```

Common interview angle: "Why is the second loop slower?" → Each `a[i][j]` in column-order lands on a different cache line; you touch a line, use one element, then move on and evict it before reusing it.

3.5 Cache Lines and Blocks

What it means: The cache doesn't store individual bytes; it stores fixed-size chunks called **cache lines** (or **blocks**), typically **64 bytes** on modern x86/ARM.

Why it matters:

- A single miss loads the **entire line**, exploiting spatial locality.
- Line size is a tradeoff: **bigger lines** → better spatial locality but more wasted bandwidth and more conflict/pollution; **smaller lines** → less waste but more misses and overhead.
- It's the unit of coherence too - crucial for **false sharing** (Section 3.13).

Address breakdown: A physical address is split for cache lookup:

| Tag | Index | Block Offset |

- **Block offset:** which byte within the line (for 64-byte line → 6 bits).
- **Index:** which set/line to look in.
- **Tag:** the remaining high bits, stored to verify the exact block.

Example: With 64-byte lines, addresses **0x1000** to **0x103F** all live in the same line. Touch one, and all 64 bytes come along.

Interview angle: "What's a cache line and why 64 bytes?" and "How is an address mapped to a cache location?" (know the tag/index/offset split).

3.6 Cache Hit and Miss

What it means:

- **Hit:** requested data is in the cache → fast.
- **Miss:** not in cache → fetch from the next level (slower), and load the line into cache.

Key metrics:

- **Hit rate** = hits / total accesses. **Miss rate** = 1 – hit rate.
- **Miss penalty** = extra cycles to service a miss.
- Small changes in miss rate matter a lot because miss penalty is huge (10-100x hit time).

Why it matters: Performance is dominated by miss rate × miss penalty (see AMAT). Going from 95% to 99% hit rate can *halve* effective access time.

Example: 1000 accesses, 950 hits (1 cycle), 50 misses (100 cycles):

Total = $950 \times 1 + 50 \times 100 = 950 + 5000 = 5950$ cycles
 AMAT = $5950 / 1000 = 5.95$ cycles

The 5% misses cause ~84% of the total time.

Interview angle: "Define hit rate and compute AMAT." Also: "Why can a 99% hit rate still be a performance problem?" (Because the 1% miss penalty may dominate.)

3.7 Direct-Mapped Cache

What it means: Each memory block maps to **exactly one** cache line, chosen by:

$$\text{line index} = (\text{block address}) \bmod (\text{number of lines})$$

Why it matters: Simplest and fastest to look up (only one place to check → one tag comparison). But it suffers heavily from **conflict misses**: two hot blocks that map to the same line kick each other out repeatedly, even when the rest of the cache is empty.

Example: 4-line cache. Blocks 0, 4, 8 all map to line 0. A loop alternating between blocks 0 and 4 thrashes line 0 forever → miss every time, despite lines 1-3 sitting idle.

Diagram:

```
Memory blocks:  0  1  2  3  4  5  6  7
Cache lines:    0  1  2  3  0  1  2  3   (block mod 4)
Block 0 and 4 both fight over line 0.
```

Interview angle: "What's a conflict miss and which mapping suffers most?" → Direct-mapped.

3.8 Fully Associative Cache

What it means: A memory block can go in **any** cache line. To find data, you compare the tag against **all** lines simultaneously.

Why it matters:

- **No conflict misses** - maximum flexibility, best hit rate for a given size.
- **Expensive:** needs a comparator for every line (parallel search via CAM - content-addressable memory) and a full replacement policy. Doesn't scale to large caches.

Where used: Small structures where flexibility matters most - e.g., the **TLB** is often fully or highly associative.

Example: A 4-line fully associative cache can hold blocks 0, 4, 8, 12 all at once - no forced eviction. Direct-mapped couldn't.

Interview angle: "Why isn't the whole cache fully associative?" → Cost/power/latency of comparing all tags in parallel.

3.9 Set-Associative Cache

What it means: The practical middle ground. The cache is divided into **sets**; each set has **N lines** (N-way). A block maps to **one set** (like direct-mapped), then can go in **any of the N lines** within that set (like fully associative).

```
set index = (block address) mod (number of sets)
```

Why it matters: Captures most of the benefit of associativity (few conflict misses) at a fraction of the cost. Real L1/L2/L3 caches are typically **4-way to 16-way** set-associative.

Special cases:

- **1-way set-associative = direct-mapped.**
- **N-way where N = total lines = fully associative.**

Example (2-way): Blocks 0 and 4 map to the same set, but the set has 2 lines, so **both fit** - no thrashing. A third conflicting block (8) triggers a replacement decision within that set.

Diagram:

```
2-way set-associative, 2 sets:
Set 0: [ line ][ line ]  <- blocks 0,2,4,6 map here
Set 1: [ line ][ line ]  <- blocks 1,3,5,7 map here
```

Interview angle: "Compare direct-mapped, set-associative, fully associative" (see comparison table in Section 8). Know that associativity reduces conflict misses with diminishing returns past ~8-way.

3.10 Cache Replacement Policies

What it means: When a set (or fully associative cache) is full and a new block must come in, which existing line do we **evict**?

Common policies:

Policy	Idea	Pros	Cons
LRU (Least Recently Used)	Evict the line unused for longest	Exploits temporal locality well	Expensive to track exactly for high associativity
FIFO	Evict the oldest-loaded line	Simple	Ignores reuse; can evict a hot line
Random	Evict a random line	Dead simple, cheap HW	Unpredictable, occasionally bad
LFU (Least Frequently Used)	Evict least-accessed	Good for skewed access	Needs counters; slow to adapt
Pseudo-LRU (PLRU)	Approximate LRU with a bit tree	Cheap, close to LRU	Not exactly LRU

Why it matters: For direct-mapped, there's no choice (one line). For associative caches, the policy directly affects miss rate. Real hardware uses **pseudo-LRU** because true LRU is too costly at 8/16-way.

Example (LRU, 2-way set): Access order to a set: A, B, A, C.

- Load A, load B (set full). Access A → A is now MRU, B is LRU.
- Access C → miss, evict **B** (LRU). Set now holds {A, C}.

Belady's anomaly (advanced): With **FIFO**, adding *more* cache can sometimes *increase* misses. LRU and other "stack" policies don't suffer this.

Interview angle: "Simulate LRU on this access sequence" and "Why do real CPUs use pseudo-LRU instead of true LRU?"

3.11 Write-Through vs Write-Back

What it means: What happens to lower memory levels when the CPU **writes** to a cached location?

- **Write-through:** Write updates cache **and** main memory immediately. Memory is always current.
- **Write-back:** Write updates only the cache; the line is marked **dirty**. Memory is updated **later**, when the line is evicted.

Why it matters: This is a classic bandwidth vs simplicity/consistency tradeoff.

	Write-Through	Write-Back
Memory traffic	High (every write)	Low (only on eviction)
Complexity	Simple	Needs dirty bit + writeback logic
Data consistency w/ memory	Always consistent	Memory can be stale
Performance	Slower on writes	Faster (absorbs repeated writes)
Typical use	Some L1 designs, simple systems	Most modern caches (L2/L3)

Dirty bit: In write-back, each line has a dirty bit. On eviction: if dirty → write to memory first; if clean → just discard.

Write buffer: Write-through caches usually add a **write buffer** so the CPU doesn't stall waiting for the memory write to finish.

Example: A counter incremented a million times:

- Write-through → a million memory writes (slow).
- Write-back → the value stays in cache; memory is written **once** on eviction.

Interview angle: "Which is faster and why?" and "What is a dirty bit for?" Common follow-up: "How does write-back complicate cache coherence?" (memory isn't the source of truth → other caches must snoop).

3.12 Write-Allocate vs No-Write-Allocate

What it means: What happens on a **write miss** (writing to an address not currently in cache)?

- **Write-allocate (fetch-on-write):** Load the block into cache first, then write to it. Bets you'll access it again.
- **No-write-allocate (write-around):** Write straight to memory, **don't** load into cache.

Why it matters: Determines whether writes pollute or warm the cache.

Common pairings (this is the key insight):

- **Write-back + write-allocate** (most common): repeated writes to the same block stay in cache and are cheap.
- **Write-through + no-write-allocate:** writes go around the cache; sensible because write-through already writes to memory anyway.

Example: Zeroing a large array you'll immediately reuse → write-allocate is good (block gets cached). Writing a huge buffer you'll never read again (streaming write) → no-write-allocate avoids evicting useful data.

Interview angle: "Pair the write-hit policy with the write-miss policy." Expected answer: write-back↔write-allocate, write-through↔no-write-allocate, and *why*.

3.13 Cache Miss Types (The 3 C's + 1)

What it means: A taxonomy of *why* a miss happened.

Type	Cause	How to reduce
Compulsory (Cold)	First-ever access to a block; it can't be cached yet	Prefetching, larger block size
Capacity	Working set larger than the whole cache	Bigger cache, better locality (blocking/tiling)
Conflict (Collision)	Blocks map to the same set and evict each other, though cache isn't full	Higher associativity, better data layout
Coherence (4th C)	A block was invalidated by another core's write (multicore)	Reduce sharing, fix false sharing

Why it matters: Diagnosing which C dominates tells you how to fix a slow program:

- Lots of compulsory → prefetch.
- Capacity → improve locality (loop blocking).
- Conflict → change layout or associativity.
- Coherence → fix false sharing / reduce contention.

Example: A 2 MB matrix multiply on a 1 MB cache thrashes → **capacity** misses. Loop **tiling/blocking** shrinks the working set to fit → dramatic speedup.

Interview angle: "Name the 3 C's" and "You profiled a hot loop and see high miss rate - how do you tell if it's capacity vs conflict?" (Capacity: fixed by more cache; conflict: fixed by associativity/layout without more cache.)

3.14 Multi-Level Caches (L1/L2/L3)

What it means: Multiple cache levels between CPU and RAM, each larger and slower than the one above.

Why it matters: A single cache can't be both fast and big. L1 optimizes for **latency** (tiny, split into instruction/data, per-core); L3 optimizes for **capacity/hit-rate** (big, shared across cores).

Design points:

- **L1** is usually **split** into **L1-I** (instructions) and **L1-D** (data) - a *Harvard* split - so fetch and load/store don't contend.
- **L2** is often per-core and unified.
- **L3** is large and **shared** among all cores (the "Last-Level Cache", LLC).

Inclusion policy:

- **Inclusive:** L2/L3 contains copies of everything in L1. Simplifies coherence (snoop only the LLC) but wastes capacity.
- **Exclusive:** A block lives in only one level. Maximizes effective capacity but complicates lookups.
- **NINE** (Non-Inclusive Non-Exclusive): neither guaranteed.

Example: Data flow on an L1 miss: check L2 → hit → copy line to L1 and serve CPU. On L2 miss → L3 → RAM.

Interview angle: "Why not just one big fast cache?" and "Inclusive vs exclusive - tradeoff?" and "Why split L1 into instruction and data?"

3.15 Cache Coherence Basics

What it means: In a multicore system, each core has its own cache. If two cores cache the **same** memory line and one writes to it, the other must not keep reading a **stale** copy. **Coherence** is the guarantee that all cores see a consistent view of memory.

Why it matters: Without it, multithreaded programs would read garbage. It's implemented in hardware and is a major cost/complexity source in multicore design.

MESI protocol - the classic coherence protocol. Each cache line is in one of:

State	Meaning
Modified	This cache has the only, dirty copy; memory is stale
Exclusive	This cache has the only, clean copy; matches memory
Shared	Multiple caches may hold this clean copy
Invalid	Line is stale/unused

Variants: **MOESI** (adds **O**wned), **MESIF** (adds **F**orward, used by Intel).

Snooping vs Directory:

- **Snooping:** Every cache watches ("snoops") a shared bus for others' reads/writes. Simple, but bus doesn't scale to many cores.

- **Directory-based:** A directory tracks which caches hold each line; used in large/NUMA systems. Scales better.

False sharing (must-know): Two cores update **different variables** that happen to sit on the **same cache line**. Coherence hardware bounces the line back and forth (invalidations) even though the data is logically independent → severe slowdown.

```
struct { int a; int b; } s; // a and b likely same 64-byte line
// Core 1 writes s.a in a loop, Core 2 writes s.b in a loop
// -> line ping-pongs between caches every write = false sharing
```

Fix: pad/align so hot per-thread variables live on separate lines (e.g., `alignas(64)`).

Interview angle: "Explain MESI." "What is false sharing and how do you fix it?" (This is one of the highest-value multicore interview topics.)

3.16 Memory Interleaving

What it means: Main memory is split into multiple independent **banks/modules**, and consecutive addresses are spread **across** them (e.g., address 0→bank0, 1→bank1, 2→bank2, 3→bank3, 4→bank0...). While one bank is busy servicing an access, others can work in parallel.

Why it matters: DRAM has latency you can't hide, but interleaving improves **throughput/bandwidth**: sequential accesses (very common due to spatial locality) hit different banks and pipeline instead of serializing on one busy bank.

Types:

- **Low-order interleaving:** low address bits pick the bank → consecutive addresses spread out (great for sequential streams). Most common.
- **High-order interleaving:** high bits pick the bank → contiguous blocks stay in one bank (better fault isolation, worse for streaming).

Example: Reading a large array sequentially across 4 interleaved banks lets 4 accesses be in flight at once → up to ~4x effective bandwidth vs a single bank.

Real hardware: This is the idea behind **multi-channel memory** (dual/quad-channel) and DRAM bank-level parallelism.

Interview angle: "How does interleaving improve memory performance?" (bandwidth/parallelism, not single-access latency).

3.17 Virtual Memory Interaction

What it means: Programs use **virtual addresses**; the **MMU** translates them to **physical addresses** via **page tables**. Caches and virtual memory interact closely because the CPU issues virtual addresses but caches usually store physical data.

Key pieces:

- **Pages:** memory managed in fixed blocks (commonly 4 KB).
- **Page table:** maps virtual pages → physical frames (per process).
- **TLB (Translation Lookaside Buffer):** a small, fast cache **of translations** so you don't walk the page table on every access. A **TLB miss** triggers a page-table walk (slow). A **page fault** means the page isn't in RAM at all → fetch from disk (very slow).

Cache addressing (the subtle interview point):

- **PIPT** (Physically Indexed, Physically Tagged): translate first, then look up cache. Simple, no aliasing, but translation is on the critical path.
- **VIPT** (Virtually Indexed, Physically Tagged): index the cache with virtual bits **while** the TLB translates in parallel, then compare physical tags. Common for L1 - hides translation latency. Constrains L1 size/associativity to avoid aliasing.
- **VIVT** (Virtually Indexed, Virtually Tagged): fastest but suffers **aliasing/homonym** problems; rare.

Why it matters: Virtual memory gives isolation, protection, and the illusion of large contiguous memory. The TLB is itself a cache in the hierarchy, and TLB misses can rival cache misses for cost.

Example: Accessing `arr[i]`: virtual address → TLB lookup (hit → physical frame) → cache lookup. If TLB misses, hardware walks the page table (multiple memory accesses) before the data access even begins.

Interview angle: "What's a TLB?" "Difference between a TLB miss and a page fault?" "Why is L1 often VIPT?"

3.18 Non-Uniform Memory Access (NUMA)

What it means: In multi-socket (and some large multicore) systems, memory is **physically distributed**: each CPU/socket has its own **local** memory. Accessing local memory is fast; accessing another socket's **remote** memory is slower (goes over an interconnect like Intel UPI / AMD Infinity Fabric). "Non-uniform" = access time depends on *which* memory you touch.

Contrast with **UMA** (Uniform Memory Access / SMP), where all CPUs share one memory with equal latency.

Why it matters: On big servers, ignoring NUMA can cost 30-100%+ performance. Software (and the OS) should keep a thread's data in its **local** node.

OS/software techniques:

- **First-touch allocation:** a page is placed in the node of the thread that first writes it → allocate data on the core that uses it.
- **Thread/memory affinity** (`numactl`, `pthread_setaffinity_np`) to pin threads and memory together.
- NUMA-aware allocators and schedulers.

Example: A thread on socket 0 repeatedly reading an array allocated on socket 1's memory pays remote-access latency every miss. Allocating that array locally (first-touch on socket 0) removes the penalty.

Interview angle: "What is NUMA and why does thread placement matter?" and "How does the OS reduce remote memory access?" (first-touch, affinity).

3.19 Hardware Prefetching

What it means: Hardware **predicts** which data you'll need next and fetches it into cache **before** you ask - turning would-be misses into hits by hiding latency.

Common prefetchers:

- **Next-line prefetcher:** on a miss to line N, also fetch line N+1.
- **Stride prefetcher:** detects a constant stride (e.g., accessing every 8th element) and prefetches ahead by that stride.
- **Stream prefetcher:** detects sequential streams and runs ahead of the access.

Why it matters: Prefetching mainly attacks **compulsory** and some capacity/latency misses. It's why simple sequential loops run near memory-bandwidth limits. But **bad prefetching hurts:** it can pollute the cache and waste bandwidth if predictions are wrong (e.g., pointer-chasing / random access).

Software prefetch: Compilers/programmers can insert explicit hints (`__builtin_prefetch`, `PREFETCH` instruction) for irregular patterns the hardware can't predict.

Example: Summing an array sequentially - the stride/next-line prefetcher stays ahead, so the CPU rarely stalls. A linked-list traversal (random pointers) defeats hardware prefetching → each node is a likely miss.

Interview angle: "How does prefetching reduce misses?" "Why doesn't it help linked lists / pointer chasing?" (unpredictable addresses). "When can prefetching hurt?" (cache pollution, wasted bandwidth).

4. Real-World Example

Optimizing a matrix multiply (the canonical case)

Naive matrix multiply $C = A \times B$ is slow not because of arithmetic but because of **cache misses** on **B** (accessed column-wise = poor spatial locality) and a working set that exceeds cache = **capacity misses**.

Naive:

```
for (i = 0; i < N; i++)
  for (j = 0; j < N; j++)
    for (k = 0; k < N; k++)
      C[i][j] += A[i][k] * B[k][j]; // B[k][j] strides down a column -> misses
```

Cache-blocked (tiled): Process the matrices in small sub-blocks that fit in cache, so each loaded block is fully reused before eviction (maximizing temporal + spatial locality, minimizing capacity misses):

```
for (ii = 0; ii < N; ii += B)
  for (jj = 0; jj < N; jj += B)
    for (kk = 0; kk < N; kk += B)
      for (i = ii; i < ii+B; i++)
        for (j = jj; j < jj+B; j++)
```

```
for (k = kk; k < kk+B; k++)
    C[i][j] += A[i][k] * B[k][j];
```

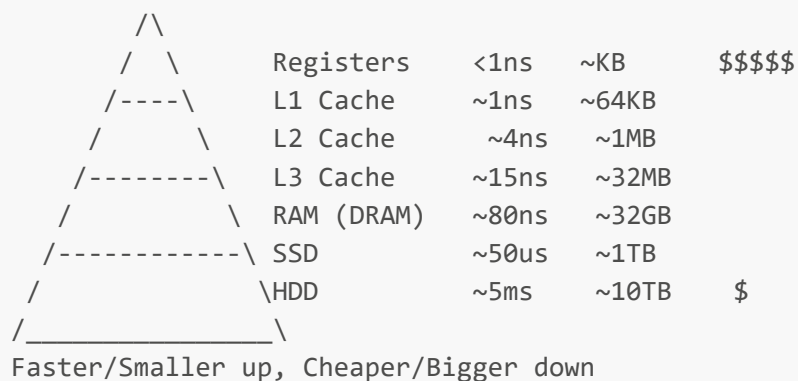
Same math, **often 5-10x faster** purely from better cache behavior. This is why BLAS libraries (used in every ML framework) are heavily cache-tiled.

Other everyday appearances

- **Databases:** buffer pool (RAM) caches disk pages; B-tree nodes sized to disk blocks; query engines tune for CPU cache.
- **Browsers:** HTTP cache, and V8 laying out object properties for cache-friendly access.
- **Backend/Redis:** Redis is literally "keep hot data in RAM instead of hitting the slow disk DB" - the hierarchy pattern applied at the app level.
- **CDNs:** edge caches (fast/near) in front of origin servers (slow/far) - the memory hierarchy scaled to the internet.
- **OS:** page cache, TLB, swap - the OS manages the RAM↔disk boundary exactly like hardware manages cache↔RAM.

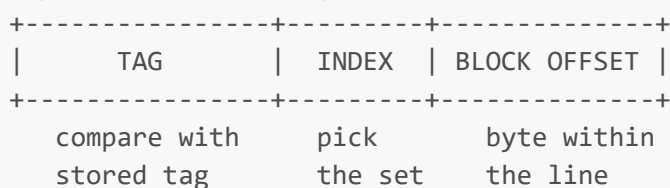
5. Diagrams / Mental Models

The pyramid (speed/size tradeoff)



Address decode for cache lookup

Physical address (e.g., 32 bits):



Mapping strategies at a glance

```

Direct-mapped:      block -> exactly ONE line
Fully associative:  block -> ANY line
N-way set assoc:   block -> one SET, any of N lines in it

```

Cache access flow (decision tree)

```

CPU request address X
|
In L1? --yes--> return (~1-4 cyc)
| no
In L2? --yes--> load to L1, return (~10-15 cyc)
| no
In L3? --yes--> load up, return (~40 cyc)
| no
In RAM? --yes--> load up, return (~200 cyc)
| no (page not in RAM)
PAGE FAULT --> read from disk (~ms), OS loads page

```

MESI state summary

```

      read hit      write
I --> (fetch) --> S/E -----> M
S --(other core writes)--> I
M --(other core reads)---> S (write back first)

```

6. Common Interview Questions

Q1. What is the memory hierarchy and why does it exist?

- **Answer:** A layered storage arrangement trading speed for cost/capacity - registers, cache, RAM, storage. It exists because fast memory is small/expensive and large memory is slow/cheap; the hierarchy gives the illusion of a large fast memory by exploiting locality.
- **Interviewer expects:** the speed/cost/size tradeoff and the word *locality*.
- **Common mistake:** listing levels without explaining *why* (the tradeoff) or forgetting locality is the enabler.

Q2. Explain temporal vs spatial locality with examples.

- **Answer:** Temporal = same location reused soon (loop counter, accumulator). Spatial = nearby locations accessed soon (array traversal, struct fields). Caches exploit both: temporal by keeping recent data, spatial by loading whole lines.
- **Expected:** one concrete code example each.
- **Mistake:** swapping the two, or giving vague examples.

Q3. What is a cache line? Why load a whole line on a miss?

- **Answer:** A fixed chunk (usually 64 B) that's the unit of transfer/storage. Loading the whole line exploits spatial locality - neighbors are likely accessed next, so it's a free prefetch.
- **Expects:** 64 bytes, spatial locality, tag/index/offset awareness.
- **Mistake:** thinking caches store single bytes/words.

Q4. Compare direct-mapped, set-associative, and fully associative caches.

- **Answer:** Direct-mapped: one line per block, fast lookup, most conflict misses. Fully associative: any line, no conflict misses, expensive (compare all tags). Set-associative: block→set, N lines/set - the practical compromise (typically 4-16 way).
- **Expects:** conflict-miss tradeoff, cost of associativity, that 1-way=direct and all-way=fully.
- **Mistake:** confusing "set" with "line," or claiming fully associative is used for large caches.

Q5. Direct-mapped vs set-associative for conflict misses - which is worse and why?

- **Answer:** Direct-mapped, because two hot blocks mapping to the same line evict each other repeatedly even with the rest of the cache empty. Associativity gives alternative lines in the set.
- **Expects:** a thrashing example.
- **Mistake:** confusing conflict misses with capacity misses.

Q6. Explain write-through vs write-back. When use each?

- **Answer:** Write-through writes cache+memory every time (simple, consistent, high traffic). Write-back writes only cache, marks dirty, flushes on eviction (fast, low traffic, memory can be stale). Modern caches mostly use write-back; write-through suits simple/consistency-critical designs.
- **Expects:** dirty bit, traffic tradeoff, coherence implication.
- **Mistake:** forgetting the dirty bit and eviction writeback.

Q7. What are the 3 C's of cache misses?

- **Answer:** Compulsory (cold, first access), Capacity (working set > cache), Conflict (mapping collisions in an under-full cache). Plus a 4th, Coherence, in multicore.
- **Expects:** how to reduce each (prefetch / bigger cache or blocking / associativity / less sharing).
- **Mistake:** confusing capacity and conflict.

Q8. What is cache coherence and what is false sharing?

- **Answer:** Coherence keeps all cores' caches consistent for shared lines (e.g., MESI). False sharing is when independent variables share a line, so writes by different cores bounce the line back and forth via invalidations, killing performance. Fix by padding/aligning to separate lines.
- **Expects:** MESI states, the ping-pong description, `alignas(64)` fix.
- **Mistake:** confusing coherence (hardware, cache-line consistency) with consistency/memory-ordering models.

Q9. What is the TLB and how does it relate to caches?

- **Answer:** The TLB is a cache of virtual→physical address translations, avoiding a page-table walk per access. It's part of the hierarchy: a TLB miss walks the page table; a page fault fetches from disk. L1 caches are often VIPT so translation overlaps cache indexing.

- **Expects:** TLB miss $\neq$ page fault, VIPT idea.
- **Mistake:** conflating TLB miss with cache miss or page fault.

Q10. How do you compute Average Memory Access Time (AMAT)?

- **Answer:** $AMAT = \text{Hit time} + \text{Miss rate} \times \text{Miss penalty}$, nested for multiple levels. Then plug in numbers (be ready to compute a 2-3 level example).
- **Expects:** correct nesting and arithmetic.
- **Mistake:** using global miss rate where local is needed (L2 miss rate is *relative to L1 misses*).

Q11. Why is iterating a 2D array row-major faster than column-major in C?

- **Answer:** C stores arrays row-major (contiguous rows). Row-order traversal walks contiguous memory $\rightarrow$ strong spatial locality $\rightarrow$ cache-line reuse. Column-order jumps a full row each step $\rightarrow$ new line each access $\rightarrow$ thrashing.
- **Expects:** spatial locality, cache-line concept, and awareness that Fortran is the opposite (column-major).
- **Mistake:** blaming the CPU/compiler instead of memory layout.

Q12. What is hardware prefetching and when does it fail?

- **Answer:** Hardware predicts future accesses (next-line, stride, stream) and loads them early, hiding latency. It fails on unpredictable patterns like pointer chasing / random access, and can hurt via cache pollution and wasted bandwidth.
- **Expects:** why linked lists defeat it, downside of over-prefetching.
- **Mistake:** claiming prefetching helps all access patterns.

7. Deep-Dive Questions

D1. Inclusive vs exclusive multi-level caches - tradeoffs? Inclusive (LLC holds all of L1/L2) simplifies coherence - snoops only check the LLC to know if any core caches a line - but wastes capacity (data duplicated). Exclusive maximizes effective capacity (each block in one level) but complicates lookups and coherence, and needs data movement between levels on hits. Intel historically favored inclusive LLC; AMD often uses exclusive/victim-cache designs. NINE is a middle ground.

D2. Why do real CPUs use pseudo-LRU instead of true LRU, and what's Belady's anomaly? True LRU requires ordering all N ways per set - $O(N \log N)$ state and updates per access - too costly/power-hungry at 8/16-way. Pseudo-LRU (a tree of bits) approximates it cheaply with near-LRU hit rates. Belady's anomaly: with **FIFO**, increasing cache size can *increase* misses on some sequences; "stack" policies like LRU are provably immune because a larger cache always contains the smaller cache's contents.

D3. How does write-back interact with cache coherence? Under write-back, the freshest data can live in a cache (Modified state), not memory - so memory is not the source of truth. Coherence must therefore find and supply the modified copy: on another core's read of an M line, the owning cache must **write back / forward** the data and downgrade to Shared. This is why MESI needs the M state and cache-to-cache transfers; a pure write-through design would keep memory current but flood the bus.

D4. Explain VIPT caches and the aliasing constraint. VIPT indexes the cache with virtual address bits **in parallel** with TLB translation, then compares the **physical** tag - hiding translation latency on the L1 critical path. The catch: if index bits come from the *translated* part of the address, two virtual addresses mapping to

the same physical page could land in different sets (aliasing). To avoid it, the index+offset must fit within the page offset (untranslated bits), which caps L1 size to $\text{page_size} \times \text{associativity}$. That's a big reason L1 caches stay small (e.g., 32 KB, 8-way, 4 KB pages).

D5. On a NUMA machine, how do you get near-local memory performance, and why does malloc-then-parallel-init matter? Use **first-touch**: pages are placed on the node whose thread first *writes* them. If the main thread initializes a big array serially, all pages land on one node → every other socket pays remote latency. Instead, initialize in parallel with the same thread/loop partitioning you'll use later, and pin threads (affinity) so each thread's data is local. Combine with NUMA-aware allocators and interleaved placement for shared data. This is why many HPC apps "touch" memory with the parallel loop before real computation.

8. Comparison Tables

Cache mapping strategies

Feature	Direct-Mapped	Set-Associative (N-way)	Fully Associative
Block placement	One fixed line	One set, N choices	Any line
Tag comparisons	1	N	All lines
Conflict misses	High	Low	None
Hardware cost	Lowest	Medium	Highest
Lookup speed	Fastest	Fast	Slowest
Replacement policy	Not needed	Within set	Across whole cache
Typical use	Rare today	L1/L2/L3 (real CPUs)	TLB, small buffers

Write policies

	Write-Through	Write-Back
Writes go to	Cache + memory	Cache only (dirty bit)
Memory traffic	High	Low
Memory freshness	Always current	May be stale
Complexity	Low	Higher (writeback logic)
Usual partner (write miss)	No-write-allocate	Write-allocate
Modern usage	Occasional L1	Dominant

Write-miss policies

	Write-Allocate	No-Write-Allocate
On write miss	Load block, then write	Write to memory, skip cache
Good for	Data reused after write	Streaming/one-shot writes

	Write-Allocate	No-Write-Allocate
Pairs with	Write-back	Write-through

SRAM vs DRAM

Feature	SRAM (cache)	DRAM (main memory)
Cell	6 transistors	1 transistor + 1 capacitor
Speed	Very fast	Slower
Refresh	Not needed	Needs periodic refresh
Density	Low	High
Cost/bit	High	Low
Use	Registers, caches	Main memory

Miss types (3 C's + 1)

Type	Cause	Primary fix
Compulsory	First access	Prefetch, bigger blocks
Capacity	Working set > cache	More cache, loop blocking
Conflict	Mapping collision	More associativity, layout
Coherence	Invalidated by another core	Reduce/fix sharing

UMA vs NUMA

Feature	UMA (SMP)	NUMA
Memory access time	Uniform for all CPUs	Local fast, remote slow
Scalability	Limited (shared bus)	Scales to many sockets
Programming	Simpler	Needs affinity/first-touch
Example	Small multicore desktop	Multi-socket servers

Cache addressing schemes

Scheme	Index by	Tag by	Pro	Con
PIPT	Physical	Physical	No aliasing	Translation before lookup
VIPT	Virtual	Physical	Overlaps TLB, fast	Size/assoc constrained
VIVT	Virtual	Virtual	Fastest	Aliasing/homonym issues

9. Common Mistakes

1. **"Cache stores individual bytes."** No - it stores **cache lines** (~64 B). The line is the unit of transfer and coherence.
 2. **Confusing a "set" with a "line."** A set contains N lines (N-way). A block maps to one *set*, then any *line* within it.
 3. **Mixing up capacity and conflict misses.** Capacity: even a fully associative cache of that size would miss. Conflict: the cache isn't full but mapping forces eviction (won't happen in fully associative).
 4. **Thinking higher associativity is always better.** Returns diminish past ~8-way; it costs power, area, and latency. 8-16 way is the sweet spot.
 5. **Confusing write-back's dirty bit with the valid bit.** Valid = line holds real data; dirty = line was modified and differs from memory.
 6. **Swapping temporal and spatial locality.** Temporal = *time* (same address again). Spatial = *space* (nearby addresses).
 7. **Assuming memory is the source of truth in a write-back multicore system.** The freshest copy may be Modified in some core's cache.
 8. **Confusing TLB miss, cache miss, and page fault.** TLB miss → page-table walk. Cache miss → next memory level. Page fault → page not in RAM, fetch from disk (OS involved).
 9. **Believing prefetching helps everything.** It fails on random/pointer-chasing patterns and can pollute the cache.
 10. **Confusing cache coherence with memory consistency.** Coherence = single-location, keep caches in sync. Consistency = ordering rules *across* multiple locations/operations.
 11. **Ignoring NUMA - assuming all RAM is equally fast** on multi-socket machines.
 12. **Using global vs local miss rate incorrectly in AMAT.** L2's miss rate in the formula is relative to accesses that *reach* L2 (i.e., L1 misses).
-

10. Edge Cases / Special Cases

- **Direct-mapped = 1-way set-associative; fully associative = "all-ways" set-associative.** Same math, extremes of the spectrum.
- **Belady's anomaly:** more cache can mean *more* misses under FIFO (not under LRU/stack policies).
- **False sharing:** correct code, terrible performance - the classic "why is my parallel code slower than serial?" trap. Fix with padding/alignment.
- **Cache line straddling / unaligned access:** a value crossing a line boundary touches **two** lines → two potential misses; misaligned atomics can be very costly or split.
- **Write-back eviction cost:** evicting a dirty line forces a memory write *before* the new line loads - a miss can trigger two memory transactions.
- **TLB shutdown:** when the OS changes a mapping, it must invalidate other cores' TLB entries via inter-processor interrupts - expensive.
- **Huge pages (2 MB/1 GB):** fewer TLB entries cover more memory → fewer TLB misses, but more internal fragmentation.
- **Cold cache after context switch:** a newly scheduled thread finds caches/TLB full of another process's data → burst of compulsory-like misses ("cache pollution").
- **VIPT size ceiling:** L1 can't grow freely without more associativity because of the aliasing constraint (index must stay within page offset).
- **Non-temporal / streaming stores:** instructions that bypass the cache on purpose for write-once data, to avoid evicting useful lines.

- **Inclusive-cache back-invalidation:** evicting a line from an inclusive LLC forces eviction from the L1/L2 above it too.

11. How to Explain in Interview

"The memory hierarchy exists because we can't have memory that's simultaneously fast, big, and cheap. So we layer it: tiny fast registers and SRAM caches near the CPU, larger slower DRAM as main memory, and huge slow disks below. It works because programs have **locality** - they reuse recent data (temporal) and access nearby data (spatial). So hardware keeps hot data in fast levels and moves data in whole **cache lines** (~64 bytes) to exploit spatial locality.

On an access, we check L1, then L2, L3, then RAM - each level filters most requests, so **average access time** stays close to L1 speed even though RAM is 100x slower. Caches map addresses via tag/index/offset and are usually **set-associative** to balance speed against conflict misses. Writes use **write-back** with a dirty bit to cut memory traffic. In multicore, **coherence protocols like MESI** keep caches consistent, and a subtle trap there is **false sharing**. Higher up, the **TLB** caches address translations, and on big servers **NUMA** means local memory is faster than remote."

Keep it to that arc: *why* → *locality* → *levels* → *mapping* → *writes* → *coherence* → *VM/NUMA*. Then let them drill in.

12. Quick Revision Notes

Key definitions

- **Memory hierarchy:** speed-vs-cost-vs-size layered storage.
- **Locality:** temporal (reuse in time), spatial (reuse in space).
- **Cache line/block:** unit of caching, ~64 B.
- **Hit/miss:** data present/absent in cache.
- **AMAT = Hit time + Miss rate × Miss penalty** (nested for multilevel).
- **Dirty bit:** line modified, not yet written to memory.
- **TLB:** cache of virtual→physical translations.

Important points

- Levels: Registers → L1 → L2 → L3 → RAM → SSD → HDD (faster/smaller/costlier up).
- Address = **Tag | Index | Offset**.
- Mapping: direct (1 line), set-assoc (N lines/set), fully-assoc (any line).
- Real caches: **write-back + write-allocate, 8-16 way**, L1 split I/D + VIPT.
- Miss types: **Compulsory, Capacity, Conflict** (+ Coherence).
- Coherence: **MESI**; watch **false sharing** (fix with 64-byte alignment/padding).

Common comparisons

- Direct vs set vs fully associative.
- Write-through vs write-back; write-allocate vs no-write-allocate.
- SRAM vs DRAM; UMA vs NUMA; PIPT vs VIPT.
- TLB miss vs cache miss vs page fault.

Must-remember facts

- Cache line ~64 bytes. Page ~4 KB.
- 1-way = direct-mapped; all-ways = fully associative.
- Write-back pairs with write-allocate; write-through with no-write-allocate.
- Row-major (C) fast; column-major slow - spatial locality.
- Prefetching fails on pointer chasing / random access.

Interview traps

- Set $\neq$ line. Capacity $\neq$ conflict. Temporal $\neq$ spatial.
- Coherence $\neq$ consistency. TLB miss $\neq$ page fault.
- Memory isn't the source of truth under write-back.
- Belady's anomaly only for FIFO.

13. Practice Tasks

1. **Compute AMAT.** Given L1 (hit 2, miss 4%), L2 (hit 12, miss 25% of L1 misses), memory 200 cycles - compute AMAT. (Answer $\approx 2 + 0.04 \times (12 + 0.25 \times 200) = 2 + 0.04 \times 62 = 4.48$.)
2. **Trace LRU.** 2-way set, access sequence to one set: A B C A B D. Mark each hit/miss and show set contents after each. (Practice eviction ordering.)
3. **Decode an address.** 32-bit address, 32 KB cache, 64-byte lines, 8-way set-associative. Compute: #sets, offset bits, index bits, tag bits. (Sets = $32768 / (64 \times 8) = 64 \rightarrow 6$ index bits, 6 offset bits, 20 tag bits.)
4. **Measure locality in code.** Write a program that sums an $N \times N$ matrix both row-major and column-major; time both for large N and explain the gap using spatial locality.
5. **Cause and fix false sharing.** In C++/Rust, have two threads increment two adjacent counters in a struct; measure the slowdown, then add `alignas(64)` padding and re-measure.
6. **Simulate a direct-mapped cache.** In Python, model a direct-mapped cache: given a stream of addresses, count hits/misses; then extend to set-associative with LRU and compare hit rates.
7. **Loop blocking.** Take a naive matrix multiply, add cache tiling, and benchmark the speedup for $N = 1024$.
8. **Classify misses.** For a given access trace, categorize each miss as compulsory / capacity / conflict (hint: compare against a fully associative cache of the same size to isolate conflict misses).
9. **Explore prefetching.** Compare summing an array (sequential) vs traversing a shuffled linked list of the same size; explain the performance gap via prefetching and spatial locality.
10. **NUMA experiment (if you have a multi-socket box or use numactl).** Allocate + init an array serially vs with parallel first-touch, then run a parallel read; measure the difference.

14. Final Cheat Sheet

Core definition: The memory hierarchy layers storage by speed/cost/size (registers → caches → RAM → disk) to give the illusion of large, fast, cheap memory - made possible by **locality of reference**.

Why it matters: Real program performance is dominated by memory access, not CPU arithmetic. Understanding the hierarchy is how you explain and fix slow code.

Must-remember numbers/facts:

- Cache line $\approx$ 64 B; page $\approx$ 4 KB.
- Levels: Reg → L1($\sim$ 1ns) → L2 → L3 → RAM($\sim$ 80ns) → SSD → HDD($\sim$ 5ms).
- Address = Tag | Index | Offset.
- Direct(1-way) → Set-assoc(N-way, real caches use 8-16) → Fully-assoc.
- Real caches: **write-back + write-allocate**, L1 split & VIPT.
- Misses: **Compulsory, Capacity, Conflict** (+Coherence). Coherence: **MESI**.

Most-asked questions: temporal vs spatial locality; direct vs set vs fully associative; write-through vs write-back; the 3 C's; AMAT computation; false sharing; TLB vs page fault; why row-major beats column-major.

Common comparisons: mapping strategies · write policies · write-allocate vs not · SRAM vs DRAM · UMA vs NUMA · PIPT vs VIPT · TLB miss vs cache miss vs page fault.

One-line interview answer:

"The memory hierarchy trades speed for capacity across registers, caches, RAM, and disk, and it works because programs exhibit temporal and spatial locality - so hardware keeps hot data in small fast levels, moves data in cache-line-sized blocks, and keeps average access time close to L1 speed even though main memory is $\sim$ 100x slower."